Lab 4 Report

The Art of Communication in MPI

Huzaifa Arshad

Contents

Chapter 1

Introduction

1.1 Objective

The objective of this lab is to understand MPI communication mechanisms, including point-to-point and collective operations such as Broadcast, Reduce, Scatter, Gather, and Scan. The lab also evaluates performance differences between manual and built-in implementations.

Chapter 2

Conceptual Foundations

2.1 Latency vs Bandwidth

Latency refers to the time required for a message to travel from sender to receiver. Bandwidth refers to the volume of data that can be transmitted per unit time. Small messages are latency dominated, while large messages are bandwidth dominated.

2.2 Blocking vs Non-blocking

Blocking communication halts execution until completion, which may lead to inefficiency. Non-blocking communication allows overlapping of computation and communication, improving performance.

2.3 MPI_Reduce Tree Mechanism

MPI_Reduce operates using a tree-based structure where processes combine values in pairs, reducing total communication steps to $\log N$.

2.4 Overlapping Communication and Computation

Blocking communication prevents overlapping, whereas non-blocking allows concurrent execution of communication and computation.

Chapter 3

Performance Experiments

3.1 Manual Broadcast vs MPI_Bcast

MPI_Bcast is more efficient because it uses optimized tree-based communication rather than sequential sending.

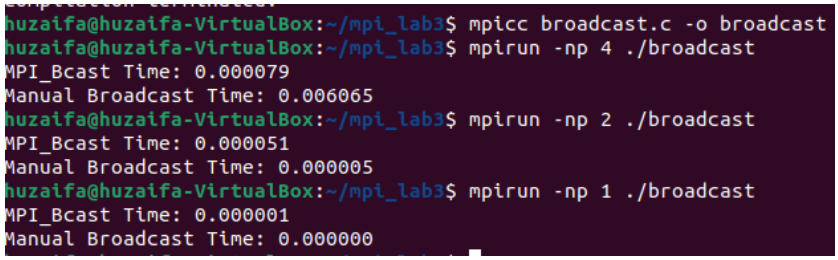

Figure 3.1: Manual Broadcast vs MPI_Bcast

3.2 Tree-Based Reduce

Tree-based reduction improves efficiency by reducing communication steps.

```
huzaifa@huzaifa-VirtualBox:~/mpi_lab3$ nano reduce.c
huzaifa@huzaifa-VirtualBox:~/mpi_lab3$ mpicc reduce.c -o reduce
huzaifa@huzaifa-VirtualBox:~/mpi_lab3$ mpirun -np 1 ./reduce
Final Reduced Sum = 1
huzaifa@huzaifa-VirtualBox:~/mpi_lab3$ mpirun -np 2 ./reduce
Final Reduced Sum = 3
huzaifa@huzaifa-VirtualBox:~/mpi_lab3$ mpirun -np 4 ./reduce
Final Reduced Sum = 10
```

Figure 3.2: Tree Reduce Output

3.3 Scatter Operation

MPI_Scatter distributes data across processes efficiently.

```
huzaifa@huzaifa-VirtualBox:~/mpi_lab3$ nano scatter.c
huzaifa@huzaifa-VirtualBox:~/mpi_lab3$ mpicc scatter.c -o scatter
huzaifa@huzaifa-VirtualBox:~/mpi_lab3$ mpirun -np 1 ./scatter
MPI_Scatter Time: 0.000004
huzaifa@huzaifa-VirtualBox:~/mpi_lab3$ mpirun -np 2 ./scatter
MPI_Scatter Time: 0.000023
huzaifa@huzaifa-VirtualBox:~/mpi_lab3$ mpirun -np 4 ./scatter
MPI_Scatter Time: 0.000036
```

Figure 3.3: Scatter Output

Chapter 4

Ring Allgather

The ring algorithm enables each process to share its data with all others in $size - 1$ steps.

```
huzaifa@huzaifa-VirtualBox:~/mpi_lab3$ nano ring.c
huzaifa@huzaifa-VirtualBox:~/mpi_lab3$ mpicc ring.c -o ring
huzaifa@huzaifa-VirtualBox:~/mpi_lab3$ mpirun -np 4 ./ring
Rank 0: 1 1 1 4
Rank 1: 1 2 1 4
Rank 2: 1 2 3 4
Rank 3: 1 1 3 4
huzaifa@huzaifa-VirtualBox:~/mpi_lab3$ mpirun -np 2 ./ring
Rank 0: 1 2
Rank 1: 1 2
huzaifa@huzaifa-VirtualBox:~/mpi_lab3$ mpirun -np 1 ./ring
Rank 0: 1
```

Figure 4.1: Ring Allgather Output

Chapter 5

Prefix Sum Computation

5.1 Linear Prefix Sum

Each process sequentially accumulates values from previous processes.

```
huzaifa@huzaifa-VirtualBox:~/mpi_lab3$ nano linear.c
huzaifa@huzaifa-VirtualBox:~/mpi_lab3$ mpicc linear.c -o linear
huzaifa@huzaifa-VirtualBox:~/mpi_lab3$ mpirun -np 1 ./linear
Rank 0: Prefix Sum = 1
huzaifa@huzaifa-VirtualBox:~/mpi_lab3$ mpirun -np 2 ./linear
Rank 0: Prefix Sum = 1
Rank 1: Prefix Sum = 3
huzaifa@huzaifa-VirtualBox:~/mpi_lab3$ mpirun -np 4 ./linear
Rank 0: Prefix Sum = 1
Rank 1: Prefix Sum = 3
Rank 2: Prefix Sum = 6
Rank 3: Prefix Sum = 10
```

Figure 5.1: Linear Prefix Sum

5.2 Using MPI_Scan

MPI_Scan provides an optimized implementation of prefix sum.

```
huzaifa@huzaifa-VirtualBox:~/mpi_lab3$ nano mpiscan.c
huzaifa@huzaifa-VirtualBox:~/mpi_lab3$ mpicc mpiscan.c -o mpiscan
huzaifa@huzaifa-VirtualBox:~/mpi_lab3$ mpirun -np 4 ./mpiscan
Rank 1: Prefix Sum = 3
Rank 0: Prefix Sum = 1
Rank 3: Prefix Sum = 10
Rank 2: Prefix Sum = 6
huzaifa@huzaifa-VirtualBox:~/mpi_lab3$ mpirun -np 2 ./mpiscan
Rank 0: Prefix Sum = 1
Rank 1: Prefix Sum = 3
huzaifa@huzaifa-VirtualBox:~/mpi_lab3$ mpirun -np 1 ./mpiscan
Rank 0: Prefix Sum = 1
```

Figure 5.2: MPI Scan Output

Chapter 6

Distributed Prefix Sum

Each process computes local prefix sums and adjusts them using global offsets obtained through MPI_Scan.

```
huzaifa@huzaifa-VirtualBox:~/mpi_lab3$ nano distributedPrefixSum.c
huzaifa@huzaifa-VirtualBox:~/mpi_lab3$ mpicc distributedPrefixSum.c -o distribut
edPrefixSum
huzaifa@huzaifa-VirtualBox:~/mpi_lab3$ mpirun -np 1 ./distributedPrefixSum
Rank 0: 1 3 6 10
huzaifa@huzaifa-VirtualBox:~/mpi_lab3$ mpirun -np 2 ./distributedPrefixSum
Rank 0: 1 3 6 10
Rank 1: 15 21 28 36
huzaifa@huzaifa-VirtualBox:~/mpi_lab3$ mpirun -np 4 ./distributedPrefixSum
Rank 3: 91 105 120 136
Rank 0: 1 3 6 10
Rank 1: 15 21 28 36
Rank 2: 45 55 66 78
```

Figure 6.1: Distributed Prefix Sum

Chapter 7

Conclusion

MPI collective operations significantly outperform manual implementations due to optimized communication strategies such as tree-based and ring-based algorithms. This lab demonstrates the importance of selecting appropriate communication techniques for scalable parallel computing.